

Prompt 工程记录

课程作业三：用 Vibe Coding 打造「我的数据小馆」个人主页

工具：Claude Code (Anthropic)

记录人：Yuyukocutie (Azygoatcoder)

一、整体策略

采用渐进式 Prompt 策略，分 5 个阶段推进：

- 需求分析 — 让 AI 先读作业文档，理解评分标准
- 创意构思 — 用视觉选项卡 + 选择题逐步确定方向
- 设计确认 — 每个模块的布局用 mockup 确认后再编码
- 一次成型 — 单次 Prompt 写出完整 HTML，避免碎片化
- 迭代打磨 — 根据实际效果逐项修复和增强

核心原则：不一次性堆需求，让 AI 每次只做一件事并确认后再继续。

二、关键 Prompt 对话记录

Prompt 1：需求导入

请阅读 homework-requirements 中的作业要求，看 vibe_coding.docx 就行，
我们已经在 Claude Code 中工作了

AI 输出：读取了 docx 文件，提取出完整的作业要求和评分标准，并触发了 brainstorming skill。

修改思路：让 AI 先理解评分标准（完整性 25 + 个性化 25 + 视觉创意 20 + Prompt 工程 20 + 反思 10），确保后续设计对齐评分维度。

Prompt 2：视觉风格选择

你想要什么视觉风格？
A 极简白灰 B 莫兰迪色系 C 暗色科技风 D 暖色调生活

我的选择：C（暗色科技风）

设计思路：暗色风格与 AI 编程主题天然契合，终端/IDE 的视觉语言能让页面更有“技术人”辨识度，避免和同学撞风格。

Prompt 3: 数据模块选择

作息/步数, 本学期课表+主观打分, 个人的AI编程经历与token用量,
背景音乐, 大学时间轴 (目前大二下)

AI 反应: 认为"AI编程经历+token用量"这个角度独特, 呼应作业主题。

修改思路: 5 个数据模块覆盖面广——生活 (作息)、学业 (课表)、技术 (AI编程)、兴趣 (音乐)、成长 (时间轴), 能全面展示个人特征。

Prompt 4: 彩蛋选择

日夜主题切换, 头像互动, 2048小游戏, 页面特效, 音乐播放器

AI 反应: 逐一列出每个彩蛋的触发方式和实现方案, 全部融入 IDE 界面。

修改思路: 彩蛋不做孤立按钮, 而是自然融入界面——主题切换在标题栏、2048藏在文件树、粒子作为背景层。

Prompt 5: 布局方案

A (终端 IDE 仪表盘)

选择左侧文件树 + 右侧卡片网格的 IDE 仪表盘方案。AI 展示了 mockup 线框图, 我确认后开始编码。

Prompt 6: Bug 反馈 (关键 Prompt)

日间背景下字体颜色辨识度不够, 看不清

AI 翻车点: 最初设计的亮色主题使用了 `<font color="#d63384">#4c4f69</font>` 作为文字色、`<font color="#d63384">#8c8fa1</font>` 作为次要文字色, 在 `<font color="#d63384">#eff1f5</font>` 背景上对比度严重不足。

修复方案: AI 将文字色加深到 `<font color="#d63384">#1e1e2e</font>` (对比度 ~13:1), 次要文字改为 `<font color="#d63384">#5c5f77</font>` (~6:1), 强调色从淡蓝 `<font color="#d63384">#89b4fa</font>` 改为浓蓝 `<font color="#d63384">#1e66f5</font>`。用 Playwright 验证修复效果。

Prompt 策略反思: 与其说"颜色不好看", 不如直接说"辨识度不够, 看不清"——具体描述问题现象比描述感受更有效。

Prompt 7: 功能扩展

你觉得还有什么功能是值得加入的?

AI 建议:

1. 终端命令面板 (敲 `<font color="#d63384">whoami</font>`、`<font color="#d63384">neofetch</font>` 输出信息)
2. 技能工具栈 (进度条展示技能水平)
3. 页面加载动画 (终端启动序列)

我的回复: `<font color="#d63384">可以</font>` (全部采纳)

修改思路: 让 AI 主动推荐功能, 比自己提需求更高效。AI 基于已有设计风格推荐的功能通常比自己拍脑袋的更贴合。

Prompt 8: 用户驱动的功能建议

我建议再给终端加上刷新功能 (这样终端更整洁)

AI 实现: 在终端卡片标题栏加了 `<font color="#d63384">□ 刷新</font>` 按钮, 同时让 `<font color="#d63384">clear</font>` 命令也恢复到初始欢迎状态。

Prompt 策略反思: 这次是我主动提需求, 说明用户驱动的 Prompt 比纯被动接受 AI 建议更有价值。

Prompt 9: 二次功能扩展

你觉得还有什么功能是值得加入的?

AI 建议:

1. 贡献热力图 (GitHub 风格 52×7 格子)
2. 打字机效果 (bio 逐字打出)

我的选择: `<font color="#d63384">可以做1和2</font>`

Prompt 10: 拉取真实 GitHub 数据

建议我们拉取本人GitHub的真实用户记录, <https://github.com/Azzygoatcoder>, 这是我自己, 然后可以更新Prompt工程记录

AI 实现: 通过 GitHub API (`<font color="#d63384">api.github.com/users/Azzygoatcoder</font>`) 拉取了完整的用户资料、仓库列表和近期活动事件。

提取到的真实数据:

- 个人信息: Yuyukocutie, 上海交通大学智能感知专业, 大二

- 原创项目: `research-pilot` — 多智能体 AI 论文阅读与实验设计系统 (Python, 1 star)

- 核心贡献: `sjtu-agent` 项目的主要贡献者, 发布了 v0.2.0-rc8 至 v0.3.0 共 4 个版本, 提交了 AI HOT 资讯推送、飞书共享客户端重构、tools.py 拆分 (5269 行 → 7 个子模块, -32%) 等功能

- 技术栈: Python、LLM、多智能体、RAG、FastAPI、飞书集成、Rust

- 近期活跃度: 近 3 天内 20+ 次 push/PR/release 事件

Prompt 策略反思: 用 GitHub API

拉取真实数据, 比手动填写更有说服力, 也体现了"数据必须真实属于你"的作业要求。AI 可以直接解析 API JSON 并映射到页面的数据模块中。

Prompt 11: 更新真实课表

本周真实课表:周一14:00-15:40 嵌入式系统与接口技术 周二08:00-10:45 信号与系统(B类)
14:00-17:40 人工智能算法实践 周三08:00-10:45 人工智能理论与应用 周四12:55-15:40
马克思主义基本原理 16:00-17:40 嵌入式系统与接口技术 周五08:00-09:40
工程实践与科技创新II 16:00-17:40 大学物理(A类)(3)

AI 实现: 将 `DATA.courses` 数组替换为 8 门真实课程, 包含课程名、时间、学分和评分。

Prompt 策略反思: 直接提供原始数据 (课表时间) 比让 AI 生成更高效, 确保数据 100% 真实。

Prompt 12: 头像替换与背景音乐

可以换成这个头像, 加入适合的音频(可以进行下载, 你肯定懂这个图像对应的)

AI 实现:

1. 将用户提供的 `Image.png` 复制为 `avatar.png`, 替换 emoji 头像
2. 添加 `bgm.mp3` 背景音乐, 实现真实的音频播放器 (支持播放/暂停/进度条/切换)
3. 更新播放器 UI 为真实音频控制 (不再模拟播放)

Prompt 策略反思: 这次交互体现了"隐含信息"的处理——用户说"你肯定懂这个图像对应的", AI 需要根据上下文 (用户名 Yuyukocutie) 推断角色和对应音乐。

Prompt 13: GitHub Pages 部署

接下来部署到GitHub Pages

AI 实现:

1. 初始化 Git 仓库, 提交 index.html、avatar.png、bgm.mp3
2. 通过 `gh repo create` 创建公开仓库
`Azzygoatcoder/personal-page`
3. 通过 GitHub API 启用 GitHub Pages (master 分支, 根目录)

部署地址: <https://azzygoatcoder.github.io/personal-page/>

Prompt 策略反思: 部署是加分项 (+3-5 分), 应在代码完成后一次性完成。CLI 比手动操作 GitHub 网页更高效。

三、踩坑复盘

坑 1: `` 标签丢失导致页面空白

现象: 新增三个功能 (boot 序列、技能条、终端面板) 后, Playwright 测试发现整个页面 body 为空, 所有元素都无法找到。

排查过程:

- 检查文件编码、BOM、null 字节 → 正常
- 检查 CSS 花括号平衡 → 正常
- 最终发现: Edit 工具在追加 CSS 时, 闭合标签被吞掉, 浏览器把整个 HTML 当成 CSS 解析

修复: 在 `</head>` 前手动补回 `</style>`。

教训: 大规模 CSS 追加后, 务必检查 `</style>` 是否完好。可以用 `grep '</style>'` 快速验证。

坑 2: 终端 `resetTerminal` 空引用

现象: 刷新终端时报 `Cannot set properties of null (setting 'textContent')`。

原因: 先执行 `resetTerminal()` 再执行 `document.getElementById('boot-time').textContent = now`, 再执行 `output.innerHTML = ...`。第一次调用后 `boot-time` 元素被 innerHTML 覆盖销毁, 第二次调用时 `getElementById` 返回 null。

修复: 把时间直接嵌入模板字符串, 不再依赖 DOM 元素。

教训: `<font color="#d63384">innerHTML = ...</font>` 会销毁所有子元素的引用, 之后不能再通过 ID 访问它们。

坑 3: GBK 编码导致 Playwright 输出崩溃

现象: Windows 终端使用 GBK 编码, emoji 字符 (🐼、🐼🐼) 无法输出, 触发 `<font color="#d63384">UnicodeEncodeError</font>`。

修复: 在脚本开头添加:

```
<font color="#d63384"></font>`python
```

```
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')
```